CMIS 3153 · ADVANCED DATABASE SYSTEMS

Mid Exam Study Pack · Lesson 2

Indexing Structures

Lesson in simple English · Formula cheat box · Answer recipes · Tutorial answers · Past paper questions 2020/21 – 2023/24 with answers

INSIDE THIS PACK

- How to use this pack
- Lesson part 1: Indexes and index types
- Lesson part 2: B-trees, B+ trees and choosing indexes
- Cheat box: index formulas and rules
- How-to-answer recipes
- Tutorial 2 (Q13–Q15): B-tree and B+ tree structure
- Lab exercises: SQL for indexes (explained line by line)
- Past paper questions with answers

★ asked in more than one past paper · (25m) marks in the paper · Yellow boxes = past paper questions · Blue boxes = formulas and key notes · Green boxes = given data

Contents

How to use this pack 4

Quick-find table 4

Lesson part 1: Indexes and index types 6

1.1 What is an index? 6

1.2 Dense vs sparse indexes 6

1.3 The three kinds of single-level ordered index 6

1.4 Searching an index: the lecture numbers 7

1.5 Multilevel indexes 8

Lesson part 2: B-trees, B+ trees and choosing indexes 9

2.1 Search tree basics 9

2.2 B-tree 9

2.3 B+ tree (the one real databases use) 9

2.4 Clustered vs non-clustered indexes 10

2.5 Hash index vs tree (B+ tree) index 10

2.6 Index maintenance: cost of INSERT, DELETE, UPDATE 11

2.7 Composite and covering indexes 11

2.8 Static vs dynamic tree indexes 11

Cheat box: index formulas and rules 12

Symbols 12

Index size 12

Search cost (block reads) 12

Order of B-tree and B+ tree nodes (one node = one block) 13

B+ tree of order d (the past-paper convention) 13

Utilization and tree height 13

Lecture numbers to quote 14

How-to-answer recipes 15

Recipe 1: Insert a key into a B+ tree ★ 15

Recipe 2: Delete a key from a B+ tree ★ 15

Recipe 3: Choose indexes for a workload 15

Recipe 4: Explain how a B-tree / B+ tree speeds up a range query ★ 16

Recipe 5: "Discuss / compare" theory questions 16

Tutorial 2 (Q13–Q15): B-tree and B+ tree structure 17

Q13. Describe the general structure of a B-tree node. What is the order p of a B-tree? 17

Q14. Describe the structure of internal and leaf nodes of a B+ tree. What is the order p of a B+ tree? 17

Q15. Why is a B+ tree usually preferred to a B-tree as an access structure? 18

Lab exercises: SQL for indexes (explained line by line) 19

SQL you need for this section 19

Exercise 1: Simple index 19

Exercise 2: Composite index 20

Exercise 3: Composite index with a range query 21

Exercise 4: Performance with and without an index 21

Exercise 5: Index on a join column 22

Exercise 6: Dropping unused indexes 23

Past paper questions with answers 24

A. Purpose of indexes and index types 24

B. Clustered vs non-clustered, choosing indexes 26

C. Index maintenance and B-trees for range queries 28

D. B+ tree structure and static vs dynamic 30

E. B+ tree drawing questions ★ 31

Questions NOT in the mid scope 34

How to use this pack

Indexing has come up in **all 4 past papers**. Since 2022/23 it is usually **Question 2** (index theory) and **Question 3** (B+ tree drawings). The B+ tree drawing questions follow fixed rules, so learn the recipe and you can collect full marks.

In the exam hall:

1. Find the question type in the Quick-find table below.
2. For theory, write the points in your own words. For a B+ tree drawing, follow Recipe 1 step by step and draw each change.
3. Match the length to the marks: a **25-mark** part needs about **4 to 6 clear points**, plus a small example or table.

Signs used in this pack: ★ = asked in more than one past paper. (25m) = marks given in the paper. Yellow boxes in the print version = past paper questions.

Quick-find table

If the question asks about...	Go to
What is an index, purpose and advantages ★	Lesson part 1 · 2020/21 Q6(a), 2022/23 Q2(a)
Primary vs secondary index (with example)	Lesson part 1 · 2020/21 Q5(a)
Primary, clustering and secondary indexes, when to use each	Lesson part 1 · 2021/22 Q1(a)
Why only one primary/clustering index but many secondary	Lesson part 1 · 2021/22 Q1(b)
Dense vs sparse index	Lesson part 1
Multilevel index, how it speeds up search	Lesson part 1 · Cheat box · 2021/22 Q1(d)
Clustered vs non-clustered (unclustered) index ★	Lesson part 2 · 2020/21 Q6(b), 2022/23 Q2(b), 2023/24 Q2(b)
Choosing indexes for a workload (key, clustered?, hash or B+ tree)	Recipe 3 · 2020/21 Q5(b), Q6(c)
Hash index vs tree index	Lesson part 2 · 2020/21 Q5(b)(ii) · Lesson 1 pack (hash pros/cons)
Effect of indexes on insert, update, delete ★	Lesson part 2 · 2022/23 Q2(c)(i), Q7(e), 2023/24 Q2(c)(i)
B-tree structure and operations, B-tree for a range query ★	Lesson part 2 · 2022/23 Q2(c)(ii), 2023/24 Q2(c)(ii)
B+ tree structure, properties, balance	Lesson part 2 · Tutorial 2 Q14 · 2023/24 Q3(a)
B-tree node, order p, why B+ tree is preferred	Tutorial 2 Q13–Q15

If the question asks about...	Go to
Draw a B+ tree after insert or delete ★	Recipe 1 · 2022/23 Q3, 2023/24 Q3(c)
Utilization of a B+ tree	Cheat box · 2023/24 Q3(c)(ii)
Static vs dynamic tree index	Lesson part 2 · 2023/24 Q3(b)
SQL: CREATE INDEX, composite index, DROP INDEX, EXPLAIN	Lab exercises section

Lesson part 1: Indexes and index types

An index works like the index at the back of a textbook: instead of reading every page, you look up the word, get the page number, and go straight there. In a database, the "page number" is a **pointer to a disk block**.

1.1 What is an index?

- An **index** is an **extra file** of entries **(key value, pointer)**, kept sorted (or hashed) on one field, called the **indexing field**.
- It gives a **fast access path** to the records **without moving the records** in the data file.
- **Purpose:** find the records you want with **far fewer disk block reads**.
- An index file is much smaller than the data file (each entry is only a key + a pointer), so searching it is cheap.

Advantages of indexes:

1. **Fewer block accesses.** Lecture example (3,000-block file): linear search ≈ 1,500 reads, binary search 12, primary index 7, multilevel index 4.
2. **Fast search on fields the file is not sorted by** (secondary indexes). Many indexes can exist.
3. **Fast range queries** and sorted output with ordered / B+ tree indexes.
4. Faster **joins, ORDER BY, GROUP BY, MIN/MAX**. A **covering index** can answer a query without reading the table at all.
5. **Enforces uniqueness** (primary key or UNIQUE index).
6. Gives the **query optimizer** cheaper ways to run a query.

Costs: extra disk space, and every INSERT, DELETE and UPDATE must also update the indexes.

1.2 Dense vs sparse indexes

Dense index	Sparse (non-dense) index
One entry for **every record** (every search key value)	One entry for **some** records only, usually **one per block**
Bigger	Smaller
Works on any file, sorted or not	Works only if the data file is **sorted** on that field
Example: secondary index on a key	Example: primary index (one entry per block)

1.3 The three kinds of single-level ordered index

The kind of index depends on **two questions**: Is the data file **sorted** on this field? Is the field **unique** (a key)?

	Primary index	Clustering index	Secondary index
Field	**Ordering key** field (file is sorted on it, values unique)	**Ordering non-key** field (file is sorted on it, values repeat)	**Non-ordering** field (file is NOT sorted on it), key or non-key
One entry per...	**data block** (key of the block's first record = **block anchor**)	**distinct value** (points to the first block holding that value)	**record** (for a key field)
Dense or sparse	Sparse	Sparse	Dense (for a key)
How many per file	At most **one**	At most **one** (and not together with a primary)	**Many**
Example	Employee file sorted by Emp_ID, index on Emp_ID	Employee file sorted by Dept_No, index on Dept_No	Index on Email or Name of the employee file
Used when...	Table is stored sorted on its key and is searched or range-scanned by that key	Queries fetch **groups** of rows with the same value (all employees of Dept 5)	Fast lookups on other columns (Email, foreign keys)

> **Why only one primary or clustering index, but many secondary?** Primary and clustering indexes depend on the **physical order** of the records (they point to block anchors). A file can be **physically sorted in only one way** at a time, like books on a shelf sorted by author or by title, not both. Secondary indexes do not depend on the physical order: they have a pointer for each record wherever it is, so you can build as many as you want (limited only by space and update cost).

Simple example: STUDENT(Reg_No, Name, Email) stored sorted by Reg_No, 3 records per block.

```
Data file:  B1: 1001 1002 1003 | B2: 1004 1005 1006 | B3: 1007 1008 1009
Primary index on Reg_No (sparse, 1 per block): <1001,B1> <1004,B2> <1007,B3>
Secondary index on Email (dense, 1 per record): 9 entries, sorted by Email
```

To find Reg_No 1005: the largest anchor ≤ 1005 is 1004, so read block B2. To find an email: look it up in the secondary index, follow its pointer.

1.4 Searching an index: the lecture numbers

File: r = 30,000 records, R = 100 bytes, B = 1,024 bytes → bfr = 10, b = 3,000 blocks. Index entry = key 9 bytes + pointer 6 bytes = 15 bytes → **68 entries per index block**.

Method	Index size	Block reads
Linear search on data	–	1,500 on average
Binary search on sorted data	–	ceil(log2 3,000) = 12
Primary index (3,000 entries, 1 per block)	ceil(3,000 / 68) = 45 blocks	ceil(log2 45) + 1 = 6 + 1 = **7**

Method	Index size	Block reads
Dense secondary index (30,000 entries)	ceil(30,000 / 68) = 442 blocks	ceil(log2 442) + 1 = 9 + 1 = **10**
Multilevel index (442 → 7 → 1 blocks)	3 levels	3 + 1 = **4**

("+ 1" = one more read for the data block itself.)

1.5 Multilevel indexes

- A single-level index is searched with **binary search**, which only **halves** the search space each step: log2(b_i) reads.
- **Idea:** the first-level index is itself a sorted file, so **build an index on the index** (second level), then an index on that, until the **top level fits in one block**.
- Each index block holds **fo** entries (**fan-out**, usually 50–300). So each level divides the search space by **fo**, not by 2.
- A search reads **one block per level + one data block**: t + 1 reads, where t = ceil(log_fo r1).
- Example: 442 first-level blocks → ceil(442 / 68) = 7 second-level blocks → 1 top block. t = 3, so **4 reads instead of 10**.
- **Static multilevel index (ISAM):** fixed; new entries go to overflow pages; needs rebuilding.
- **Dynamic multilevel index (B-tree, B+ tree):** leaves free space in nodes and **splits/merges** nodes, so inserts and deletes never need a full rebuild. This is what real DBMSs use.

Lesson part 2: B-trees, B+ trees and choosing indexes

2.1 Search tree basics

- A **search tree** of order p: each node has up to **p pointers** and **p − 1 keys**, kept **sorted**. Keys in the subtree to the left of a key are smaller, keys to the right are bigger.
- Each node = **one disk block**, so a search reads **one block per level**.
- Problem with a plain search tree: after many inserts and deletes it becomes **unbalanced** (some paths much longer). B-trees and B+ trees fix this.

2.2 B-tree

- A **balanced** search tree: **all leaves are at the same level**.
- Every node (except the root) is **at least half full** (at least ceil(p/2) child pointers). The root has at least 2 children (unless it is the only node).
- Each key appears **once** in the tree, together with a **data pointer** to its record. Internal nodes and leaves both hold data pointers.
- **Search:** start at the root, compare the key, follow the right child, repeat down to the leaf. Cost = height of the tree (3–4 reads even for millions of rows).
- **Insert:** find the right leaf and insert in sorted order. If the node **overflows**, **split** it into two and **push the middle key up** to the parent. Splits can go up; if the root splits, a **new root** is made and the tree grows **one level taller** (it grows from the top, so it stays balanced).
- **Delete:** remove the key (an internal key is replaced by its neighbour key from a leaf). If a node **underflows** (less than half full), **borrow** a key from a sibling (redistribute) or **merge** with a sibling (pulling the separator key down). If the root becomes empty, the tree gets **one level shorter**.

2.3 B+ tree (the one real databases use)

A B+ tree is a B-tree with two changes:

1. **All data entries (key + data pointer) are in the leaves.** Internal nodes hold **only keys and child pointers** — they are just signposts that guide the search.
2. **Leaves are linked** in key order (each leaf has a pointer to the next leaf), so you can read keys in order like a sorted list.

```
                [ 30 | 60 ]                <- internal node: signposts only
        +------------+------------+
[10 20 25]-->[30 40 50]-->[60 70 80 90]    <- leaves: all keys + data pointers, linked
```

Properties: balanced (all leaves at the same depth); every node except the root at least half full; keys sorted inside nodes and across leaves; high fan-out so the tree is short (3–4 levels for millions of keys).

How a B+ tree stays balanced:

- **Insert:** if a leaf is full, **split** it; the first key of the new right leaf is **copied up** to the parent (it stays in the leaf too). If an internal node is full, split it and **push up** the middle key (it moves up, it does

not stay). If the root splits, a new root is made: the tree grows **upwards**.

- **Delete:** if a node falls below half full, first try to **redistribute** (borrow from a sibling with the same parent and update the separator key in the parent). If the sibling has no spare entries, **merge** the two nodes and remove the separator from the parent. If the root becomes empty, it is removed: the tree shrinks **from the top**.

Why balance matters: every search costs exactly the height, so search time is **predictable and logarithmic**, even after many inserts and deletes. No reorganization is needed.

Why a B+ tree is preferred over a B-tree: more keys fit in each internal node (no data pointers), so the tree is **shorter**; range queries just walk along the **linked leaves**; every search goes to a leaf, so cost is uniform; full scans read only the leaves.

2.4 Clustered vs non-clustered indexes

	Clustered index	Non-clustered (unclustered) index
Meaning	Data rows are **stored in the same order** as the index key	A **separate structure** with keys + pointers to rows stored in some other order
Per table	Only **one**	**Many**
Range query	Matching rows are **next to each other**: few, sequential page reads	Matching rows are **scattered**: up to one random read per row
Point lookup	Fast	Fast, plus one pointer hop
Insert / update cost	Higher (rows must be kept in order, page splits, row moves if key changes)	Lower on the table, but every index must be updated
Best for	Range queries, ORDER BY, GROUP BY on that column (dates, salary, IDs scanned in order)	Selective exact-match lookups (Email, Customer_ID), covering indexes

Example from Tutorial 4: a date range that matches 300,000 rows (6% of the table) is fast with a **clustered** index (rows are together), but with a **non-clustered** index it would need 300,000 random reads, so the optimizer would rather scan the whole table.

2.5 Hash index vs tree (B+ tree) index

	Hash index	B+ tree index
Exact match (=)	**Best**: about 1 bucket read	Good: height + 1 reads
Range (<, >, BETWEEN), ORDER BY, MIN/MAX, LIKE 'ab%'	**Not possible**	**Yes**
Growth	Static hashing needs rehashing; skewed data causes long chains	Grows and shrinks smoothly (always balanced)
Choose when	Column is searched **only by equality** (session ID, token)	Any ranges or sorting are needed

2.6 Index maintenance: cost of INSERT, DELETE, UPDATE

Every index is an extra copy of some data that must be kept correct.

- **INSERT:** an entry is added to **every index** on the table; a full B+ tree leaf **splits** (and splits may go up). For a clustered index the row must go into the right place in order (page splits, rows moved).
- **DELETE:** the entry is removed from every index; nodes may **underflow → redistribute or merge**.
- **UPDATE of an indexed column** = **delete the old entry + insert the new entry** in that index. If it is the clustered key, the **row physically moves**.
- **UPDATE of a column with no index:** the indexes are not touched.
- **Other costs:** extra disk space; building an index scans and sorts the whole table; **locking/contention** on busy index pages; fragmentation over time (needs rebuilds).
- **So:** index only the columns that queries really search, join or sort on; drop unused indexes; choose a stable clustered key; balance faster reads against slower writes.

2.7 Composite and covering indexes

- **Composite (multi-column) index:** an index on several columns, e.g. (customer_id, order_date). Entries are sorted by the first column, then the second.
- **Leftmost prefix rule:** the index helps queries that use the **first column** (or the first and second, and so on). An index on (customer_id, order_date) helps `WHERE customer_id = 123 AND order_date > '2023-01-01'` and `WHERE customer_id = 123`, but **not** `WHERE order_date > '2023-01-01'` alone.
- Put the **equality column first** and the **range column last**.
- **Covering index:** the index contains **all columns the query needs**, so the query is answered from the index alone, with **no table access** (much fewer I/Os).

2.8 Static vs dynamic tree indexes

- **Static tree index (ISAM style):** the upper levels are built once and **never change**; new entries go to **overflow pages**; rebuilt from time to time.
- **Dynamic tree index (B+ tree):** nodes **split and merge** on every insert and delete; always balanced.
- **Static is preferred when:** data is **read-mostly or rarely changes** (archives, reports, data warehouses, lookup tables); updates come in **periodic batches** (rebuild after each load); **many readers at once** (upper pages never change, so they need no locking); pages can be packed **100% full** (fewer pages, maybe one level less); key distribution is stable.
- **Dynamic is better when** there are frequent inserts and deletes (OLTP), because overflow chains in a static index grow long and slow.

Cheat box: index formulas and rules

Symbols

Symbol	Meaning
V	Size of the search key field (bytes)
P	Size of a block (tree) pointer (bytes)
Pr	Size of a record (data) pointer (bytes)
B	Block size (bytes)
r	Number of records in the data file; b = number of data blocks
r_i	Number of index entries; b_i = number of index blocks
fo	Fan-out = index entries per block (index blocking factor)
p	Order of a B-tree / B+ tree (max pointers per node)
d	Order as used in the past papers' B+ tree questions (min entries per node)

Index size

$$R_i = V + P$$
$$fo = bfr_i = \left\lfloor \frac{B}{R_i} \right\rfloor$$
$$b_i = \left\lceil \frac{r_i}{fo} \right\rceil$$

How many index entries (r_i)?

Index type	r_i
Primary (sparse)	number of **data blocks** b
Clustering (sparse)	number of **distinct values** of the field
Secondary on a key (dense)	number of **records** r

Search cost (block reads)

Single-level index (binary search on the index, then 1 data block):

$$\lceil \log_2 b_i \rceil + 1$$

Multilevel index (one block per level, then 1 data block). Each level = previous level divided by fo, until 1 block:

$$t = \lceil \log_{fo} r_1 \rceil$$
$$\text{reads} = t + 1$$
$$b_{j+1} = \left\lceil \frac{b_j}{fo} \right\rceil$$

Order of B-tree and B+ tree nodes (one node = one block)

$$\text{B-tree: } p \times P + (p-1)(V + Pr) \leq B$$

$$\text{B+ tree internal: } p \times P + (p-1)V \leq B$$
$$\text{B+ tree leaf: } p_{leaf}(V + Pr) + P \leq B$$

Example (Tutorial 2 style): B = 512, V = 9, P = 6, Pr = 7. Internal: 6p + 9(p − 1) ≤ 512 → 15p ≤ 521 → **p = 34**. Leaf: 16 p_leaf + 6 ≤ 512 → **p_leaf = 31**.

B+ tree of order d (the past-paper convention)

Node	Min entries	Max entries	Max pointers
Any node except the root	d	2d	2d + 1
Root	1	2d	2d + 1
d = 2 (the exam questions)	**2**	**4**	**5**

Rules to write in the exam (d = 2):

- **Leaf overflow (5 entries):** first d = 2 stay in the old leaf, the other 3 go to a new right leaf; **copy up** the first key of the new leaf into the parent.
- **Internal overflow (5 keys):** left 2 keys stay, right 2 keys go to a new node, the **middle key is pushed up** (it does not stay below).
- **Root split:** make a new root → height + 1.
- **Leaf underflow (1 entry):** borrow from a sibling with **more than d** entries (same parent) and update the separator; otherwise **merge** with the sibling and delete the separator from the parent.
- **Search rule:** in a node, go to the pointer between K(i−1) and K(i) where K(i−1) ≤ key < K(i).

Utilization and tree height

$$\text{utilization} = \frac{\text{total entries in all nodes}}{\text{number of nodes} \times \text{max entries per node}}$$

Tree height estimate: leaves = ceil(rows ÷ entries per leaf); next level = ceil(leaves ÷ fan-out); repeat until 1 node (the root). Height = number of levels. Point lookup = height reads (+ 1 for the table row if the table is separate).

Lecture numbers to quote

r = 30,000, B = 1,024, R = 100 → b = 3,000. Linear search 1,500 · binary search 12 · primary index (45 blocks) **7** · dense secondary index (442 blocks) **10** · multilevel (442 → 7 → 1) **4**.

How-to-answer recipes

Recipe 1: Insert a key into a B+ tree ★

1. **Write the rules first** (order d, min d and max 2d entries, copy up for leaves, push up for internal nodes). This alone earns marks.
2. **Search for the leaf:** at each node, follow the pointer where $K(i-1) \le key < K(i)$. Write the path (for example A → C → K).
3. **Room in the leaf?** Insert the key in sorted order. Done. Only that leaf changes.
4. **Leaf full?** Put the 2d + 1 keys in order. Keep the first d in the old leaf, move the rest to a **new leaf** to its right. **Copy up** the first key of the new leaf into the parent. Fix the leaf links.
5. **Parent full now?** Split it: keep the first d keys, move the last d keys to a new node, **push up the middle key** to the grandparent.
6. **Root split?** The pushed-up key becomes a **new root**; the height grows by 1.
7. **Draw** the changed part (or the whole tree), with node labels.

Small example (d = 2): insert 25 into a full leaf [10 20 30 40].

```
Keys in order: 10 20 25 30 40
Old leaf: [10 20]   New leaf: [25 30 40]   Copy up 25 to the parent
```

Recipe 2: Delete a key from a B+ tree ★

1. Find the leaf and remove the key.
2. **Still at least d entries?** Done. (The key may stay in an internal node as a signpost; that is allowed.)
3. **Underflow (fewer than d)?** Look at a sibling **with the same parent**.
   - Sibling has **more than d** entries → **redistribute**: move one entry across, then **update the separator key** in the parent.
   - Sibling has **exactly d** → **merge** the two leaves and **delete the separator** from the parent.
4. If the parent now underflows, repeat at that level: borrow from a sibling (the separator goes down, the sibling's key goes up), else merge by **pulling down** the separator from the grandparent.
5. If the root ends up with no keys, remove it: the merged node becomes the new root and the height drops by 1.

Recipe 3: Choose indexes for a workload

For each query pattern, answer three things in a table, then justify:

Question	How to decide
(i) Search key	The column in the WHERE condition

Question	How to decide
(ii) Clustered or not	**Clustered** if it is a range query (<, >, BETWEEN) that returns many rows, or used for sorting. **Unclustered** is enough for selective equality (= returns few rows). Only one clustered index per table.
(iii) Hash or B+ tree	**Hash** if only equality (=). **B+ tree** if any range or ordering.

Also say "no significant improvement" when a query returns most of the table, or the table is tiny.

Recipe 4: Explain how a B-tree / B+ tree speeds up a range query ★

1. Draw a small tree with sorted keys and linked leaves.
2. Describe the structure (balanced, nodes = blocks, sorted keys, half full, linked leaves).
3. Describe search, insert (split), delete (borrow / merge).
4. Walk through the query: `CREATE INDEX` on the column → search down to the first key > threshold (3–4 reads) → follow the linked leaves to collect all matching keys → fetch the rows.
5. Compare: no index = read all b blocks; with index = height + matching leaves + matching rows.

Recipe 5: "Discuss / compare" theory questions

One-line definition of each → comparison table → one concrete example (Employee, Students, Orders table) → one-line conclusion saying which is better in the given situation.

Tutorial 2 (Q13–Q15): B-tree and B+ tree structure

Q1–Q12 of Tutorial 2 are in the Lesson 1 pack.

Q13. Describe the general structure of a B-tree node. What is the order p of a B-tree?

A B-tree node looks like this (q ≤ p):

```
< P1, <K1, Pr1>, P2, <K2, Pr2>, ..., <Kq-1, Prq-1>, Pq >
```

- **P1 … Pq** = **tree pointers** to child nodes (empty / NULL in leaf nodes).
- **K1 … Kq−1** = search key values, **sorted**: K1 < K2 < … < Kq−1.
- **Pr1 … Prq−1** = **data pointers** to the record (or block) that has that key.
- Every value X in the subtree under Pi is between K(i−1) and K(i).
- **Order p** = the **maximum number of tree pointers** in a node (so at most p − 1 keys).
- Each node except the root has **at least ceil(p/2)** tree pointers; the root has at least 2 (unless it is the only node). **All leaves are at the same level.**
- p is chosen so that one node fills one disk block: p × P + (p − 1)(V + Pr) ≤ B.

Q14. Describe the structure of internal and leaf nodes of a B+ tree. What is the order p of a B+ tree?

Internal node (q ≤ p):

```
< P1, K1, P2, K2, ..., Kq-1, Pq >
```

- Holds **only keys and tree pointers** (no data pointers).
- Each internal node except the root has at least ceil(p/2) tree pointers.
- **Order p** = maximum tree pointers in an internal node; chosen so that p × P + (p − 1) × V ≤ B.

Leaf node:

```
< <K1, Pr1>, <K2, Pr2>, ..., <Kq-1, Prq-1>, Pnext >
```

- Holds **keys with data pointers**, plus **Pnext** = a pointer to the **next leaf** (the leaves form a linked list in key order).
- **Leaf order p_leaf** = maximum data entries in a leaf; each leaf holds at least ceil(p_leaf / 2); chosen so that p_leaf × (V + Pr) + P ≤ B.
- All leaves are at the same level, and **every key value appears in a leaf**.

Example: B = 512, V = 9, P = 6, Pr = 7 → internal p = 34, leaf p_leaf = 31 (see the Cheat box).

Q15. Why is a B+ tree usually preferred to a B-tree as an access structure?

1. **Higher fan-out:** internal nodes have no data pointers, so **more keys fit in each block** → **fewer levels** → fewer disk reads per search.
2. **Range queries and sequential access:** all data entries are in **linked leaves**, so after finding the first key you just follow the leaf chain. A B-tree needs an in-order walk up and down the levels.
3. **Uniform search cost:** every search ends at a leaf, so the cost is always the height.
4. **Simpler insert and delete:** data lives only in the leaves; internal keys are just copies used as signposts.
5. **Full scans** read only the leaf level.

Lab exercises: SQL for indexes (explained line by line)

These are the 6 lab exercises at the end of the Lesson 2 notes. The SQL is written for **MySQL** (XAMPP). Each statement is followed by what it means in plain English.

SQL you need for this section

Statement	What it does
`CREATE TABLE t (col type, ...)`	Makes a new table with columns and data types
`PRIMARY KEY (col)`	Marks the unique ID column; MySQL builds a (clustered) index on it automatically
`FOREIGN KEY (col) REFERENCES other(col)`	Says this column must match an ID in another table
`INSERT INTO t VALUES (...)`	Adds a row
`CREATE INDEX name ON t(col1, col2)`	Builds an index on one or more columns
`DROP INDEX name ON t`	Removes an index (MySQL form)
`EXPLAIN SELECT ...`	Shows the plan: which index (if any) the database will use

> **Reading EXPLAIN output (MySQL):** `type = ALL` means a **full table scan** (no index). `type = ref` or `range` means an **index is used**. `key` = the name of the index used. `rows` = how many rows MySQL expects to read (smaller is better).

Exercise 1: Simple index

Relation: Employee(emp_id, first_name, last_name, department, salary).

```
CREATE TABLE employees (
  emp_id      INT PRIMARY KEY,
  first_name  VARCHAR(50),
  last_name   VARCHAR(50),
  department  VARCHAR(50),
  salary      DECIMAL(10,2)
);
```

- `CREATE TABLE employees ( ... );` makes the table. Each line inside is one column: its **name** and its **data type**.
- `INT` = whole number. `VARCHAR(50)` = text up to 50 characters. `DECIMAL(10,2)` = a number with up to 10 digits, 2 after the decimal point (money).
- `PRIMARY KEY` on emp_id = each employee has a unique ID; MySQL automatically indexes it.

```
INSERT INTO employees VALUES
  (1, 'John', 'Smith', 'IT', 85000.00),
  (2, 'Nimal', 'Perera', 'HR', 60000.00);
```

- Adds rows. The values are in the same order as the columns. (The lab asks for 1,000 random rows; in practice you generate them with a script or a loop.)

```
CREATE INDEX idx_last_name ON employees(last_name);
```

- Builds an index named **idx_last_name** on the **last_name** column of employees. It is a B+ tree index (the MySQL default).

```
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
```

- `SELECT *` = all columns. `FROM employees` = from this table. `WHERE last_name = 'Smith'` = only rows where the last name is Smith.
- `EXPLAIN` in front = do not run it, just **show the plan**. You should see `key = idx_last_name` and `type = ref` (index used), not `type = ALL`.

Exercise 2: Composite index

Relation: Orders(order_id, customer_id, order_date, total_amount).

```
CREATE TABLE orders (
  order_id      INT PRIMARY KEY,
  customer_id   INT,
  order_date    DATE,
  total_amount  DECIMAL(10,2)
);

CREATE INDEX idx_customer_date ON orders(customer_id, order_date);

EXPLAIN SELECT *
FROM orders
WHERE customer_id = 123
  AND order_date > '2023-01-01';
```

- `DATE` = a date value written as 'YYYY-MM-DD'.
- The index is on **two columns**: sorted by customer_id first, then by order_date inside each customer.
- The query asks: orders of customer 123 **after** 1 Jan 2023. `AND` = both conditions must be true.
- The composite index fits perfectly: **equality on the first column** (customer_id = 123), then a **range on the second** (order_date > ...). EXPLAIN should show `key = idx_customer_date`, `type = range`.

- Note: a query with only `WHERE order_date > '2023-01-01'` could **not** use this index well (leftmost prefix rule).

Exercise 3: Composite index with a range query

Relation: Products(product_id, category, price, availability_date).

```
CREATE TABLE products (
  product_id         INT PRIMARY KEY,
  category           VARCHAR(50),
  price              DECIMAL(10,2),
  availability_date  DATE
);

CREATE INDEX idx_category_price ON products(category, price);

EXPLAIN SELECT *
FROM products
WHERE category = 'Electronics'
  AND price BETWEEN 100 AND 500;
```

- `BETWEEN 100 AND 500` = price from 100 to 500, **including** both ends.
- Index order (category, price) is right: **equality column first** (category), **range column second** (price). The database jumps to 'Electronics' and reads prices 100–500 in order.
- If the index were (price, category), the range on price would come first and the index would be used less efficiently.

Exercise 4: Performance with and without an index

Relation: transactions(transaction_id, customer_id, transaction_date, amount).

```
CREATE TABLE transactions (
  transaction_id    INT PRIMARY KEY,
  customer_id       INT,
  transaction_date  DATE,
  amount            DECIMAL(10,2)
);

-- Step 1: run the query with NO index and look at the plan
EXPLAIN SELECT *
FROM transactions
WHERE customer_id = 456
  AND transaction_date >= CURDATE() - INTERVAL 1 MONTH;

-- Step 2: add the index
CREATE INDEX idx_customer_date ON transactions(customer_id, transaction_date);

-- Step 3: run the same EXPLAIN again and compare
```

- `--` starts a **comment** (ignored by MySQL).
- `CURDATE()` = today's date. `CURDATE() - INTERVAL 1 MONTH` = the date one month ago. So the condition means "in the last month".
- **Before the index:** EXPLAIN shows `type = ALL` and `rows ≈ 10,000` (full scan).
- **After the index:** `type = range`, `key = idx_customer_date`, and `rows` is only the few matching transactions. The query runs much faster.

Exercise 5: Index on a join column

Relations: Customers(customer_id, first_name, last_name) and Orders(order_id, customer_id, order_date, total_amount), where Orders.customer_id is a foreign key.

```
CREATE TABLE customers (
  customer_id  INT PRIMARY KEY,
  first_name   VARCHAR(50),
  last_name    VARCHAR(50)
);

CREATE TABLE orders (
  order_id      INT PRIMARY KEY,
  customer_id   INT,
  order_date    DATE,
  total_amount  DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE INDEX idx_customer_id ON orders(customer_id);

EXPLAIN SELECT o.order_id, o.order_date, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.last_name = 'Smith';
```

- `FOREIGN KEY (customer_id) REFERENCES customers(customer_id)` = every order must belong to an existing customer.
- `customers c` and `orders o` give the tables **short nicknames** (aliases), so we can write `c.last_name` and `o.order_id`.
- `JOIN orders o ON c.customer_id = o.customer_id` = match each customer with **their** orders (same customer_id).
- `WHERE c.last_name = 'Smith'` = keep only customers called Smith.
- The index on orders(customer_id) lets the database, for each Smith, **jump straight to that customer's orders** instead of scanning all orders. EXPLAIN shows `key = idx_customer_id` for the orders table.

Exercise 6: Dropping unused indexes

```
DROP INDEX idx_last_name ON employees;
```

- Removes the index named idx_last_name from the employees table. (Some other databases write just `DROP INDEX idx_last_name;`, as in the lecture notes.)
- **Why drop indexes?** Every index uses disk space and **slows down INSERT, UPDATE and DELETE**, because each index must be updated too. An index that no query uses is pure cost.
- **How to check:** run EXPLAIN on your common queries before and after dropping. If the plans and speed stay the same, the index was not needed.

Past paper questions with answers

Grouped by topic. Each question shows the paper year and marks. ★ = asked in more than one paper.

A. Purpose of indexes and index types

2020/21 Q6(a) (25m) and 2022/23 Q2(a) (25m) ★ What is the main purpose of using indexes? / What is indexing? Discuss the purpose and advantages of indexing for improving query performance.

- **Definition:** an index is an **extra file of ⟨key, pointer⟩ entries**, sorted (or hashed) on an indexing field, that gives a fast **access path** to the records **without changing where the records are stored**.
- **Main purpose:** to find the records that match a search condition with **far fewer disk block reads**.
- **Advantages:**
  1. **Fewer block accesses:** in the lecture example (3,000 blocks) linear search needs about 1,500 reads, a primary index 7, a multilevel index only 4.
  2. Fast search on columns the file is **not sorted by** (secondary indexes); many indexes can exist.
  3. Efficient **range queries** and sorted output (B+ tree).
  4. Faster **joins, ORDER BY, GROUP BY, MIN/MAX**; a **covering index** answers a query without touching the table.
  5. **Enforces uniqueness** (primary key / UNIQUE).
  6. Gives the **optimizer** cheaper plans.
- **Cost (mention briefly):** extra storage, and every insert, update and delete must also update the indexes.

2020/21 Q5(a) (30m). Discuss the difference between a primary index and a secondary index, using a simple example.

- **Primary index:** built on the **ordering key field** (the field the file is physically sorted on, with unique values). It has **one entry per data block**: ⟨key of the first record in the block (block anchor), block pointer⟩. So it is **sparse**, small, and there can be **only one**.
- **Secondary index:** built on a **non-ordering field** (the file is not sorted on it). For a key field it has **one entry per record**, so it is **dense** and larger. There can be **many**.

Example: STUDENT(Reg_No, Name, Email), sorted by Reg_No, 3 records per block.

```
Data:  B1: 1001 1002 1003 | B2: 1004 1005 1006 | B3: 1007 1008 1009
Primary index on Reg_No (sparse):  <1001,B1> <1004,B2> <1007,B3>     3 entries
Secondary index on Email (dense):  <amal@..,B2> <binu@..,B1> ...     9 entries
```

To find 1005: the largest anchor ≤ 1005 is 1004 → read B2. To find binu@...: look it up in the Email index → its pointer → B1.

	Primary	Secondary
Field	Ordering key	Non-ordering field
Density	Sparse (1 per block)	Dense (1 per record)
Number per file	At most 1	Many
Size	Small	Larger
Range queries	Efficient (data is in order)	Each match may be in a different block
Lecture example (30,000 records)	45 index blocks → 7 reads	442 index blocks → 10 reads

2021/22 Q1(a) (25m). What is a primary index, a clustering index and a secondary index? When is each used?

- **Primary index:** on the **ordering key** (unique) of a sorted file; one entry per block (block anchors); sparse. **Used when** the table is stored sorted on its primary key and is often searched or range-scanned by it (Emp_ID, Order_ID).
- **Clustering index:** on an **ordering non-key field** (values repeat) of a sorted file; one entry per **distinct value**, pointing to the first block with that value; sparse. **Used when** queries fetch **groups** of rows with the same value or ranges (all employees of Dept_No 5, orders by date).
- **Secondary index:** on a **non-ordering field** (key or non-key); dense for key fields. **Used when** you need fast access on other columns the file is not sorted by (Email, Name, a foreign key like Customer_ID). Several can exist.

2021/22 Q1(b) (25m). Why can a database have at most one primary or clustering index on a file, but several secondary indexes?

- Primary and clustering indexes **depend on the physical order** of the records: their entries point to block anchors or to the first block of each value, which only works because the file is **sorted on that field**.
- A file can be **physically sorted in only one order** at a time (like books on a shelf: sorted by author **or** by title, not both). So there can be **at most one** primary **or** clustering index.
- **Secondary indexes do not depend on physical order:** each is a separate, usually dense structure with a pointer for every record, wherever it is stored. So you can create **many**, limited only by storage and the extra cost on inserts, updates and deletes.

2021/22 Q1(d) (25m). How does the multilevel indexing structure improve the efficiency of searching an index file?

- A single-level index of b_i blocks is searched by **binary search**: ceil(log2 b_i) reads, halving the search space each time.

- A **multilevel index** treats the first-level index as a sorted file and builds a **sparse index on it** (level 2), then on that (level 3), until the **top level fits in one block**.
- Each index block holds **fo** entries (fan-out, often 50–300), so each level divides the search space by **fo**, not 2.
- A search reads **one block per level + one data block**: t + 1 reads, with t = ceil(log_fo r1).
- **Example (lecture):** dense index with 30,000 entries, fo = 68, 442 blocks. Binary search = 9 + 1 = **10 reads**. Multilevel: 442 → 7 → 1 blocks, t = 3 → **4 reads**.
- **Dynamic** multilevel indexes (B-tree, B+ tree) also leave free space and split/merge nodes, so inserts and deletes never need a full reorganization.

B. Clustered vs non-clustered, choosing indexes

> **2020/21 Q6(b) (30m) and 2022/23 Q2(b) (25m) ★** Explain clustered and unclustered (non-clustered) indexes, the key differences, and examples where one is preferred.

- **Clustered index:** the **data rows are stored in the same order as the index key**. Only **one** per table. Rows for a range are **next to each other**.
- **Non-clustered index:** a **separate structure** of keys + **pointers** to rows stored in another order. **Many** per table. Matching rows may be **scattered**.

	Clustered	Non-clustered
Row order	Same as index order	Independent
Per table	One	Many
Range query I/O	Sequential, few pages	Up to one random read per row
Insert / update cost	Higher (keep order, page splits)	Lower on the table, but each index updated

- **Prefer clustered** on columns used for **ranges, sorting, grouping**: Order_Date ("orders last month"), Salary BETWEEN 40,000 AND 60,000, IDs read in order. (Tutorial 4: a 6% date range is fast with a clustered index, but with a non-clustered index it would need 300,000 random reads.)
- **Prefer non-clustered** for **selective exact-match lookups**: Email, Customer_ID (joins), Reg_No when the table is clustered on something else; and as **covering indexes**.

> **2023/24 Q2(b) (25m).** Which indexing approach best supports frequent range and exact-match queries, with reference to clustered and non-clustered indexes?

- A **B+ tree** supports **both**: exact match = one root-to-leaf search (3–4 reads); range = find the first key then follow the **linked leaves**. (A hash index supports exact match only.)
- Make the B+ tree **clustered** on the column with the **most range queries**, because ranges gain most from physical order (Tutorial 4 Ex. 3: 50,000 rows = 250 pages clustered vs up to 50,000 random reads non-clustered). Choose it carefully: only one is allowed; it should be stable and ideally ever-increasing.

- Use **non-clustered** B+ tree (or hash) indexes on **exact-match** columns with **high selectivity** (few rows per value); consider **covering** composite indexes for frequent queries.
- Remember that more indexes = slower writes.
- **Conclusion:** clustered B+ tree on the main range column + non-clustered indexes on selective exact-match columns.

> **2020/21 Q5(b) (30m).** What would you consider when choosing (i) the primary index, and (ii) hash versus tree indexes?

(i) Choice of primary (clustered) index — only one is possible, so consider:

1. Which attribute is most used for **range queries, ORDER BY, GROUP BY** or reading in order.
2. Whether the attribute is **unique** (primary index) or not (clustering index).
3. **How often it changes:** avoid columns that change often (changing them moves rows).
4. **Insert pattern:** ever-increasing keys append at the end; random keys cause page splits.
5. Use in **joins** (sorted data helps merge joins).
6. The balance of **queries vs updates** in the workload.

(ii) Hash vs tree:

- **Hash:** best for **equality (=)** only, about 1 I/O; no ranges, no ordering, no prefix search; sensitive to skewed data; static hashing needs rehashing as data grows.
- **Tree (B+ tree):** handles **equality and ranges**, sorted scans, MIN/MAX, prefix search; balanced and dynamic; cost = height + 1.
- **Decide by the queries on the attribute:** equality only → hash; any range or ordering → B+ tree.

> **2020/21 Q6(c) (45m).** R(a, b, c, d, e), at least 1000 distinct values per attribute. Workload A: 100,000 queries `select * from R where b < ?`. Workload B: 10,000 queries `select * from R where c = ?`. With up to two new indexes, give for each: (i) search key, (ii) clustered or unclustered, (iii) hash or B+ tree. Justify.

Workload	(i) Key	(ii) Clustered?	(iii) Structure
A: b < ? (range)	**b**	**Clustered**	**B+ tree**
B: c = ? (equality)	**c**	**Unclustered**	**Hash**

- **A:** a range `b < ?` can return a large part of R. With a **clustered** index the matching rows sit on consecutive pages (sequential reads); an unclustered index could need one random read per row and might be no better than a full scan. It must be a **B+ tree**, because a hash index cannot answer `<`. Big speed-up compared with scanning R 100,000 times.
- **B:** with at least 1000 distinct values, each query returns about 0.1% of R or less, so following a few pointers is cheap: **unclustered is enough**. A **hash** index is best for pure equality (about 1 bucket read); a B+ tree would also work. Big speed-up compared with a full scan per query.
- Note: if both workloads ran on the same table, the single clustered index should go to **b** (the range column).

C. Index maintenance and B-trees for range queries

> **2022/23 Q2(c)(i) (25m), 2023/24 Q2(c)(i) (25m) and 2022/23 Q7(e) (short note) ★**
> Students(Student_ID, Name, Age, GPA), often queried for GPA above a threshold / Orders(Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status), often queried for Total_Amount above a value, with continuous inserts, updates and cancellations. Discuss the impact of creating and maintaining indexes on insert, delete and update performance.

Benefit: an index on GPA / Total_Amount turns the frequent range query from a full table scan into a short index search + reading only the matching rows. UPDATE and DELETE statements with a WHERE clause also find their rows faster.

Cost: every index is an extra copy of data that must be kept correct.

Operation	What happens to the indexes
INSERT	An entry is added to **every** index in sorted position; a full B+ tree leaf **splits**, and splits may go up. For a **clustered** index the row must be placed in key order (page splits, rows moved). More indexes = slower inserts.
DELETE	The entry is removed from every index; nodes may **underflow → redistribute or merge**.
UPDATE of an indexed column	**Delete the old entry + insert the new one** (a GPA change, a Total_Amount correction). If it is the clustered key, the **row physically moves**.
UPDATE of a non-indexed column	Indexes are not touched (e.g. Order_Status change when there is no index on it).
Creating the index	Scans and sorts the whole table; uses locks and resources while building.
Other costs	Extra disk space; **locking/contention** on busy index pages; fragmentation over time → rebuilds.

Apply to the scenario:

- **Students:** GPA changes every semester → each change updates the GPA index; new students insert into all indexes; graduates are deleted. Keep **Student_ID as the clustered/primary index** and GPA as a **non-clustered B+ tree**; after a big batch of GPA updates the index can be rebuilt.
- **Orders:** continuous inserts → keep the clustered key **ever-increasing** (Order_ID / Order_Date) so new rows are added at the end without mid-table splits; use a **non-clustered B+ tree on Total_Amount** for the range query; cancellations (Order_Status updates) do not touch the Total_Amount index; avoid indexing a low-variety, frequently changing column like Order_Status.

Guidelines: index only columns that queries really filter, join or sort on; drop unused indexes; choose a stable clustered key; batch updates; check with EXPLAIN; balance read speed against write overhead.

2022/23 Q2(c)(ii) (25m) and 2023/24 Q2(c)(ii) (25m) ★ Explain how B-tree indexing speeds up the GPA queries / the Total_Amount range queries. Describe the structure and key operations of a B-tree index.

Structure:

- A **balanced search tree**: every node is a **disk block** with many **sorted keys** and pointers; **all leaves are at the same depth**.
- Order p: at most p children and p − 1 keys per node; every node except the root is **at least half full**.
- In a B-tree each key appears once with a data pointer. In a **B+ tree** (what DBMSs use) all keys with data pointers are in the **leaves**, internal nodes hold only separator keys, and the **leaves are linked**.

```
                [ 3.0 | 3.5 ]
          +-----------+-------------+
  [2.1 2.5 2.8]-->[3.0 3.2 3.4]-->[3.5 3.7 3.9 4.0]    <- linked leaves
```

Operations:

- **Search:** start at the root, compare, follow the right child, repeat to a leaf. Cost = height (3–4 reads for millions of rows).
- **Insert:** insert in the right leaf in sorted order; if it overflows, **split** and push a key up; a root split adds a new level. The tree stays balanced.
- **Delete:** remove the key; if a node underflows, **borrow** from a sibling or **merge** with it; the root may disappear (height − 1).

How it speeds up `GPA > 3.5` / `Total_Amount > 10000`:

1. `CREATE INDEX idx_gpa ON Students(GPA);` (or `ON Orders(Total_Amount)`).
2. **Search** down the tree to the first key above the threshold: about 3–4 block reads instead of reading every block.
3. **Scan forward** along the **linked leaves**, collecting every matching entry (they are already in sorted order).
4. Fetch the rows through the pointers. This is cheap when few rows match or the index is clustered. (If a huge fraction matches with a non-clustered index, the optimizer may still choose a full scan.)
5. Because the tree **stays balanced** during the frequent inserts, updates and deletes, the cost stays logarithmic and predictable.

Without an index: read all b blocks. **With the index:** height + the leaf blocks covering the range + the matching rows.

2021/22 Q1(c) (25m). Pros and cons of hash indexes. Full answer in the **Lesson 1 pack** (section G, Hashing). Short version: fastest for exact match (about 1 read) and fast inserts/deletes, but **no range queries or ordering**, collisions and skew slow it down, and static hashing needs costly rehashing.

D. B+ tree structure and static vs dynamic

2023/24 Q3(a) (25m). Explain the structure and properties of a B+ tree. How do B+ trees maintain balance, and why is this important for indexing?

Structure: a **dynamic multilevel index**; a balanced tree whose nodes are disk blocks. **Internal nodes** hold only **keys and child pointers** ⟨P1, K1, P2, ..., Kq−1, Pq⟩ that guide the search. **Leaf nodes** hold **all data entries** ⟨key, record pointer⟩ and a **next-leaf pointer**, so the leaves form a **linked list** in key order.

Properties:

1. **Balanced:** all leaves at the same depth.
2. **Occupancy:** every node except the root is at least half full (order d: d to 2d entries).
3. **Sorted** keys within nodes and across the leaves.
4. **High fan-out** → height of only 3–4 for millions of keys.
5. Every key value appears at the leaf level; internal keys are signpost copies.

Maintaining balance:

- **Insert:** a full leaf **splits** (copy up the first key of the new leaf); a full internal node splits (push up the middle key); if the root splits, a **new root** is created. The tree grows **upwards**, so all leaves stay at the same depth.
- **Delete:** an underfull node **redistributes** with a sibling (update the separator) or **merges** with it (remove / pull down the separator); if the root becomes empty it is removed. The tree shrinks **from the top**.

Why it matters:

- Every search costs **exactly the height** → guaranteed O(log N) reads and predictable response times.
- The half-full rule keeps the index **compact and short** even with heavy inserts and deletes; **no periodic reorganization** is needed (unlike sorted files or ISAM).
- **Linked leaves** make **range queries** and ordered scans efficient.

2023/24 Q3(b) (25m). When might a static tree index be preferred over a dynamic tree index?

- **Static tree index (ISAM style):** upper levels are built once and **never change**; new entries go to **overflow pages** chained from the leaves; rebuilt from time to time.
- **Dynamic tree index (B+ tree):** nodes **split and merge** on every insert/delete; always balanced.

A static index is preferred when:

1. The data is **static or read-mostly**: archives, history and reporting tables, data warehouses, lookup tables loaded in bulk. Few inserts → no long overflow chains.
2. **Updates come in periodic batches:** rebuild the index after each batch load.
3. **Many readers at the same time:** the upper pages never change, so they need **no locking**; a dynamic tree must lock nodes during splits and merges, which limits concurrency.
4. **Space matters:** a static index can be packed **100% full** (dynamic nodes are usually about 67% full), so fewer pages and sometimes one level less → faster searches.
5. The key distribution is **known and stable**; the structure is simple and predictable.

When not: frequent inserts in some key ranges create **long overflow chains**, so a dynamic B+ tree is better for busy OLTP tables.

E. B+ tree drawing questions ★

> **Rules used in all answers below (write them in the exam):** order d = 2, so every node except the root holds 2 to 4 entries. Leaf split: first 2 entries stay, the rest move to a new right leaf, and the first key of the new leaf is **copied up**. Internal split: the middle key is **pushed up**. Delete: **redistribute** with a sibling (same parent) if it has more than 2 entries, otherwise **merge**. Search: follow the pointer where K(i−1) ≤ key < K(i).

> **2022/23 Q3 (a)–(c) (20m each).** Use Figure 3.1.

```
Figure 3.1
                              A[50]
              +------------------+--------------------------+
           B[40|45]                                   C[60|78|87|90]
     +--------+-------+                +--------+--------+--------+--------+
  D[23 32] E[40 41 43 44] F[45 48]   G[50 59] H[60 66] I[78 80 85 86] J[88 89] K[96 99]
```

(a) Insert a data entry with key 90. List the nodes that are read and the nodes that are updated.

- **Nodes read: A** (90 ≥ 50 → go right to C) → **C** (90 ≥ 90 → rightmost pointer → K) → **K**.
- **Nodes updated: K only**: [96 99] becomes **[90 96 99]**. K has room (3 ≤ 4), so no split and no change to any parent.

(b) Draw the tree after inserting key 42.

1. Path: A (42 < 50) → B (40 ≤ 42 < 45) → leaf **E**.
2. E = [40 41 43 44] is full. With 42 the keys are 40 41 42 43 44 → **split**: E = [40 41], new leaf E′ = [42 43 44].
3. **Copy up 42** into B → B = [40 42 45]. B has room, so no further split. Leaf links: D → E → E′ → F.

```
                B[40|42|45]
      +--------+--------+-----------+
  D[23 32]  E[40 41]  E'[42 43 44]  F[45 48]

(A, C and the right side are unchanged)
```

(c) Draw the tree after inserting key 79.

1. Path: A (79 ≥ 50) → C (78 ≤ 79 < 87) → leaf **I**.
2. I = [78 80 85 86] is full. Keys 78 79 80 85 86 → **split**: I = [78 79], new leaf I′ = [80 85 86]. **Copy up 80** into C.
3. C becomes [60 78 80 87 90] = 5 keys → **overflow** → split the internal node: C = [60 78], C′ = [87 90], **push up 80** (the middle key) into A.
4. A becomes **[50 80]** (has room). Height stays the same.

```
                            A[50|80]
          +---------------------+----------------------+
      B[40|45]              C[60|78]                C'[87|90]
   (unchanged)          +--------+--------+      +--------+--------+
                  G[50 59] H[60 66] I[78 79]  I'[80 85 86] J[88 89] K[96 99]
```

2022/23 Q3 (d)–(e) (20m each). Use Figure 3.2.

```
Figure 3.2
                        A[50]
           +----------------+----------------+
        B[40|45]                         C[60|78]
    +-------+--------+               +--------+--------+
D[23 32] E[40 41 43 44] F[45 48]  G[50 59] H[60 66] I[78 80 85 86]
```

(d) Delete key 23 (redistribute if possible).

1. D = [23 32] → [32]: only 1 entry, less than d = 2 → **underflow**.
2. Sibling **E** (same parent B) has 4 entries (more than 2) → **redistribute**: move 40 from E into D → D = [32 40], E = [41 43 44].
3. **Update the separator** in B: the first key of E is now 41 → B = **[41 45]**.

```
          B[41|45]
    +--------+---------+
D[32 40] E[41 43 44] F[45 48]
```

(e) Delete key 50.

1. G = [50 59] → [59]: **underflow**. G's sibling under C is H = [60 66], which has exactly 2 entries → cannot lend → **merge G and H** → G = [59 60 66]. Remove the separator **60** from C → C = [78].
2. C now has only 1 key → **underflow**. Its sibling B = [40 45] has exactly 2 keys → cannot lend → **merge B and C**, **pulling down 50** from A → one node [40 45 50 78] with children D, E, F, G, I.
3. A has no keys left → it is removed; the merged node becomes the **new root**. The **height drops by one**.

```
                      [40 | 45 | 50 | 78]            <- new root
    +-----------+----------+----------+-------------+
D[23 32]  E[40 41 43 44]  F[45 48]  G[59 60 66]  I[78 80 85 86]
```

(50 can stay as a separator even though the record is deleted; writing 59 instead is also correct.)

> **2023/24 Q3(c) (40m + 10m).** B+ tree of order d = 2 (max 5 pointers, max 4 entries per node). (i) Insert one by one the keys [era, ban, bat, kin, day, log, rye, max, won, ace, ado, bug, cop, gas, let, fax] into an empty B+ tree, using dictionary (lexicographic) order. Show the tree after every 4 insertions. (ii) What is the utilization of the tree?

Step 0: sort all keys first (makes every comparison easy): ace < ado < ban < bat < bug < cop < day < era < fax < gas < kin < let < log < max < rye < won.

After insertions 1–4 (era, ban, bat, kin): one leaf, which is also the root.

```
[ban bat era kin]
```

Insertions 5–8 (day, log, rye, max):

- day: [ban bat day era kin] overflows → [ban bat] | [day era kin]; copy up **day** → new root [day].
- log: goes right → [day era kin log].
- rye: [day era kin log rye] overflows → [day era] | [kin log rye]; copy up **kin** → root [day kin].
- max: → [kin log max rye].

```
            [day | kin]
      +-----------+------------+
  [ban bat]   [day era]   [kin log max rye]
```

Insertions 9–12 (won, ace, ado, bug):

- won: [kin log max rye won] overflows → [kin log] | [max rye won]; copy up **max** → root [day kin max].
- ace: → [ace ban bat]. ado: → [ace ado ban bat].
- bug: [ace ado ban bat bug] overflows → [ace ado] | [ban bat bug]; copy up **ban** → root [ban day kin max] (4 keys: full but valid).

```
                  [ban | day | kin | max]
    +----------+-----------+-----------+------------+
[ace ado] [ban bat bug] [day era] [kin log] [max rye won]
```

Insertions 13–16 (cop, gas, let, fax): no splits.

- cop → [ban bat bug cop]. gas → [day era gas]. let → [kin let log]. fax → [day era fax gas].

```
                       [ban | day | kin | max]
    +--------------+---------------+--------------+-------------+
[ace ado] [ban bat bug cop] [day era fax gas] [kin let log] [max rye won]
```

(ii) Utilization:

- Nodes = 1 root + 5 leaves = **6** nodes. Capacity = 6 × 4 = **24** entries.
- Entries stored = root 4 + leaves (2 + 4 + 4 + 3 + 3 = 16) = **20**.
- Utilization = 20 / 24 = **0.833 = 83.3%**.

Questions NOT in the mid scope

Past paper questions on Object-Relational databases, Data Warehousing/OLAP, the three-schema architecture and NoSQL are left out, because the mid covers Lessons 1–5 only.